



The Agentic Machine Learning Engineer Project

By Jesse Anders & Nick Minnard



Main.py Layout

main.py > ...

```
1 > import argparse...
36
37 # List of agent modules
38 AGENT_MODULES = [agent1, agent2, agent2_1, agent3, agent4, agent5, agent6]
39
40 # INSTRUCTION ARCHIVE LIST: Used in the get_inst tool
41 IA_LIST = [INST_ARCHIVE, AGENT1_IA, AGENT2_IA, AGENT2_1_IA, AGENT3_IA, AGENT4_IA, AGENT5_IA, AGENT6_IA]
42
43 #=====#
44 > # region                      Dynamic Globals                      #...
58 > # region                      OpenAI API                          #...
74 > # region                      LM Studio API                       #...
110 # region                      PyFile                             #
111 #=====#
112
113 > class PyFile:...
242
243 # endregion =====#
244 > # region                      Tool Functions                      #...
780 # region                      Preprocessor                        #
781 #=====#
782
783 > class Preprocessor:...
993 # region                      FeatureEngineer                     #
994 #=====#
995
996 > class FeatureEngineer:...
1374 > # region                      File Management                     #...
1576 > # region                      Entry Point w/ Angparser           #...
```

...continued

```
243 # endregion =====#
244 # region Tool Functions #
245 #=====#
246
247 @tool
248 > def get_inst(...
249
250
251
252
253
254 @tool
255 > def logger(...
256
257
258
259 @tool
260 > def write_generated_func(...
261
262
263
264 @tool
265 > def exec_stored_func(...
266
267
268
269 @tool
270 > def exec_generated_func(...
271
272
273
274 @tool
275 > def append_to_json_list(...
276
277
278
279 @tool
280 > def add_nlp_column(...
281
282
283
284 @tool
285 > def get_pow_candidates(...
286
287
288
289 @tool
290 > def create_pow_group(...
291
292
293
294 @tool
295 > def describe_pow_group() -> str: ...
296
297
298
299 @tool
300 > def test_pow_transform(...
301
302
303
304 @tool
305 > def encode_choice(encoding_category: str, extra_info: dict = None):...
306
307
308
309 @tool
310 > def redundancy_dictionary(key_item: str, value_items: list, extra_info: dict = None):...
311
312
313
314 @tool
315 > def request_human_approval(task_to_approve): ...
316
317
318
319 @tool
320 > def request_human_exec(): ...
321
322
```

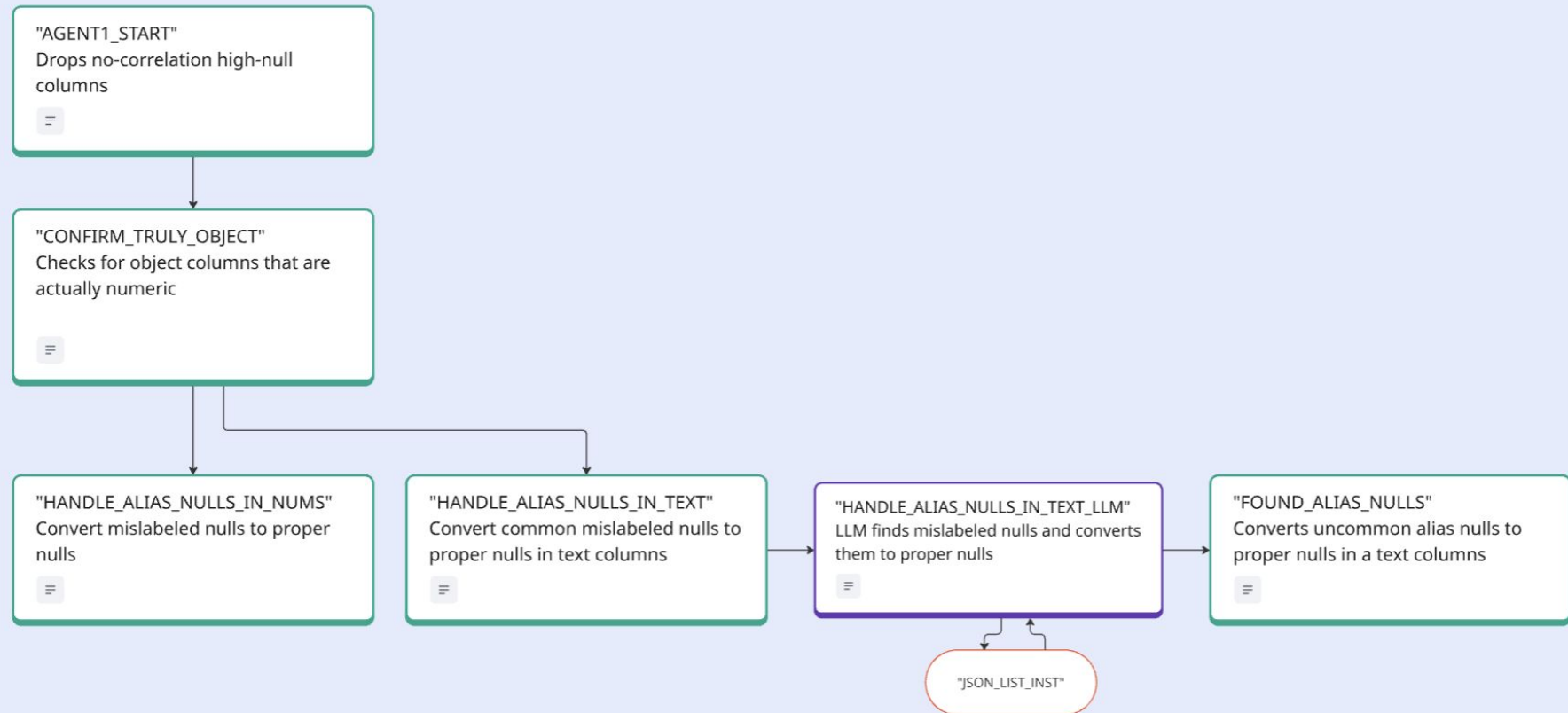
...continued

```
779 # endregion =====#
780 # region Preprocessor #
781 #=====#
782
783 class Preprocessor:
784 > def __init__(self, args, df):...
791
792 > def get_df(self):...
797
798 > def set_df(self, df):...
805
806 > def restore_backup_df(self):...
811
812 > def run(self, temperature):...
982
983 > def update_df(self, altered_df):...
991
992 # endregion =====#
993 # region FeatureEngineer #
994 #=====#
995
996 class FeatureEngineer:
997 > def __init__(self, args, df):...
1004
1005
1006 > def get_df(self):...
1011
1012 > def set_df(self, df):...
1019
1020 > def run(self, temperature):...
1363
1364 > def update_df(self, altered_df):...
```

...continued

```
main.py > FeatureEngineer > run
996     class FeatureEngineer:
1020         def run(self, temperature):
1103             # endregion =====#
1104             # region AGENT3 LOOP NLP Feature Generator #
1105             #=====#
1106             set_shared_var('current_agent_name', 'Agent 3')
1107
1108             for column in feature_engineer.get_df().columns:
1109
1110                 if self.args.run_agent_3 != True:
1111                     continue
1112
1113                 if column == self.args.target_var:
1114                     continue
1115
1116                 # DEBUGGING: Run iteration of small column set or a single column
1117                 if self.args.debug:
1118                     COLUMNS_TO_TEST = [] # Empty to Skip Agent Entirely!
1119                     if column not in COLUMNS_TO_TEST:
1120                         continue
1121
1122                 set_shared_var('current_column', column)
1123                 pipeline.write(f'set_current_column("{column}")')
1124                 inputs = {'messages': [('user', AGENT3_IA["AGENT3_START"])]}
1125                 try:
1126                     stream = agent3.stream(inputs, stream_mode='values')
1127                     print_stream(stream)
1128                 except Exception as e:
1129                     print(f'Error during stream: {e}')
1130
1131                 # 20 second delay between comlum loops to prevent tokens per minute failures.
1132                 print("20 Second pause to prevent Token Per Minute failures")
1133                 time.sleep(20)
1134
1135             save_dataframe_stage(feature_engineer.get_df(), 'POST_AGENT_3')
```

Agent 1 (Alias Nulls)



Agent 1 Instruction Archive

```
AGENT1_IA = {
#####
#   region      AGENT 1      OBJECT TO NUM AND ALIAS NULLS INSTRUCTIONS   #
#####

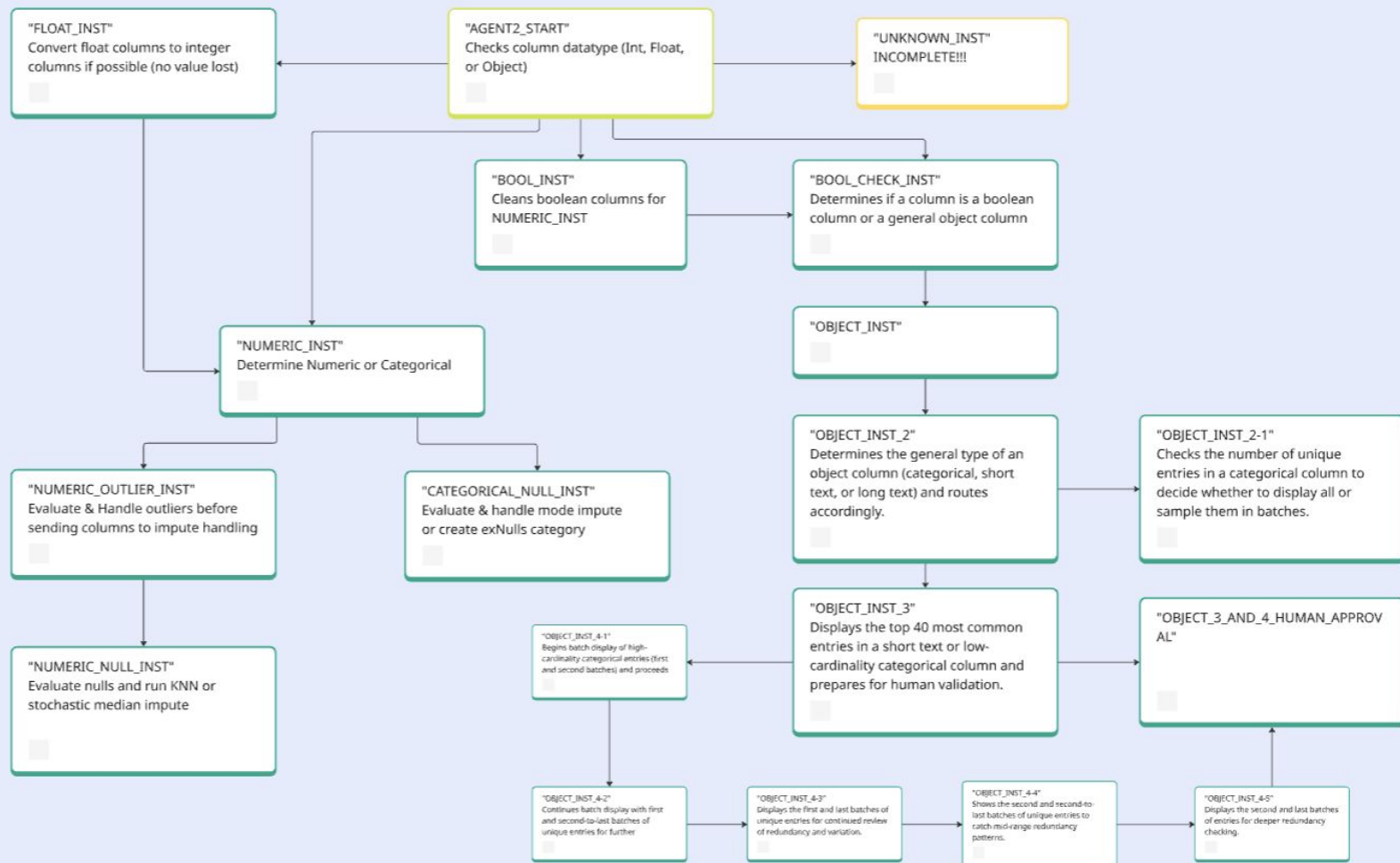
# AGENT1 Kickoff Instructions (Evaluate if High Null Percentage Dictates Column Drop)
"AGENT1_START": (
    "Use exec_stored_func tool to run evaluate_column_for_drop(df) to determine if the column should be dropped.\n"
    "If recommended_action is 'drop', exec_stored_func to run drop_column(df) to drop the column from the df. END PROCESS.\n"
    "If recommended_action is 'keep' use the exec_stored_func tool to run data_type_check(df).\n"
    "If data type is 'object' call tool get_inst(CONFIRM_TRULY_OBJECT) for further instructions.\n"
    "If data type is not 'object', END PROCESS.\n"
),
# Confirms that the column data type is truly text and not a truly numeric column with sparse text items.
"CONFIRM_TRULY_OBJECT": (
    "Use exec_stored_func tool to run check_percent_numeric(df) to determine if the column is truly object or if it is numeric.\n"
    "If column 'is less than 90% numeric', use the tool call get_inst(HANDLE_ALIAS_NULLS_IN_TEXT) for instructions.\n"
    "If column 'is 90%+ numeric and can be considered truly numeric', use exec_stored_func tool to run check_for_text_nums(df).\n"
    "If the result from check_for_text_nums comes back as True, use exec_stored_func tool to run convert_text_nums_to_numeric(df).\n"
    "Continue on by using the tool call get_inst(HANDLE_ALIAS_NULLS_IN_NUMS) for instructions.\n"
),
# Instructions for handling mislabeled or alias nulls and remaining unidentifiable text to NULL.
"HANDLE_ALIAS_NULLS_IN_NUMS": (
    "Use exec_stored_func tool to run describe_and_clean_non_numeric_entries(df) to find and convert mislabeled nulls to proper nulls.\n"
    "If 1 or more items added to the Unique Review List, use the exec_stored_func tool to run convert_all_non_num_to_null(df) to convert all remaining text entries to nulls.\n"
    "Use exec_stored_func tool to run convert_column_to_numeric(df). END PROCESS\n"
),
# Instructions for handling common mislabeled or alias nulls, like empty, unknown, none, etc in standard object type/text.
"HANDLE_ALIAS_NULLS_IN_TEXT": (
    "Use exec_stored_func tool to run convert_common_alias_nulls(df) to find and convert mislabeled nulls to proper nulls.\n"
    "Continue on by using the tool call get_inst(HANDLE_ALIAS_NULLS_IN_TEXT_LLM) for instructions.\n"
),
# Instructions for handling uncommon mislabeled or alias nulls, using LLM logic.
"HANDLE_ALIAS_NULLS_IN_TEXT_LLM": (
    "Use exec_stored_func tool to run display_most_common_unique_entries(df).\n"
    "Read the Function Output and determine if any of the listed items are 'very likely mislabeled nulls'. Some examples include 'none', 'missing', 'no data', etc.\n"
    "If no items in the Function Output are 'very likely mislabeled nulls'. END PROCESS\n"
    "If items that are very likely mislabeled nulls exist, isolate the mislabeled items as a new list and call get_inst(FOUND_ALIAS_NULLS) for instructions."
),
"FOUND_ALIAS_NULLS": (
    "For handling the specific isolated new items that are 'very likely mislabeled nulls', call get_inst(JSON_LIST_INST) for instructions on formatting a list of items.\n"
    "If you added mislabeled nulls to the list, use the exec_stored_func tool to run convert_uncommon_alias_nulls(df) to convert items in the list to nulls, then continue on by using the tool call get_inst(HANDLE_ALIAS_NULLS_IN_NUMS) for instructions.\n"
),
# Instructions for creating a well-formatted JSON list of alias nulls.
"JSON_LIST_INST": (
    "new_items SHOULD ONLY BE MISLABELED NULLS!!!"
    "Here is a simple example of a well-formatted JSON list:\n"
    '[ "na", "missing", "none", "unknown", "empty" ]\n'
    "Ensure that:\n"
    "1. Each entry is a string enclosed in double quotes.\n"
    "2. Entries are separated by commas.\n"
    "3. No trailing commas after the last item.\n"
    "4. The list should not contain any extra characters, comments, or notes.\n"
    "5. The list must be valid JSON format.\n\n"
),
}
```



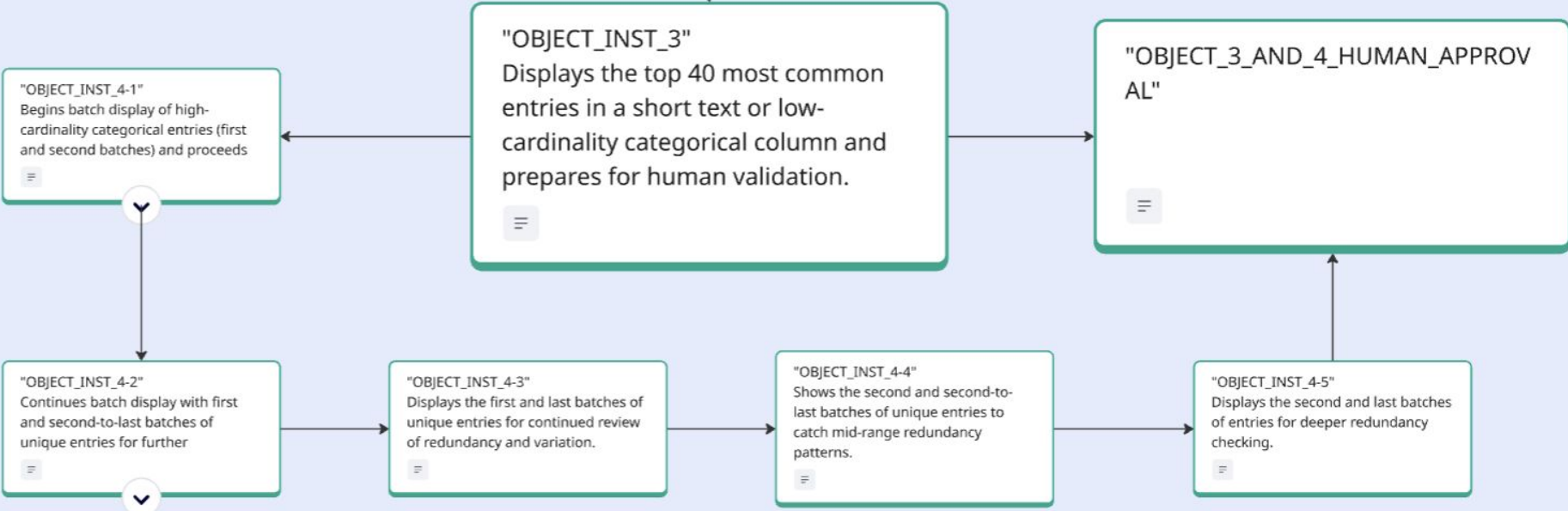
Agent 1 Static Function Library

```
3 > from word2number import w2n...
11
12 # Add the project root directory to sys.path
13 sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../..')))
14
15 # Shared State Getters
16 from utils import get_dataframe_stage, get_shared_var
17
18 # =====#
19 # region          Evaluate for Drop Null Heavy Column          #
20 #=====#
21
22 > def evaluate_column_for_drop(df, drop_null_threshold=0.5, target_corr_threshold=0.3):...
111
112
113 > def drop_column(df):...
135
136 #=====#
137 # region          COLUMN NUM AND ALIAS NULL CHECKERES          #
138 #=====#
139
140 > import re...
146
147 > def check_percent_numeric(df, numeric_threshold=0.9):...
186
187
188 > def check_for_text_nums(df):...
216
217 # PLEASE UPDATE TO INCLUDE LIST OF UPDEATED/CONVERTED ENTRIES TO NUMBERS
218 > def convert_text_nums_to_numeric(df):...
246
247
248
249 > def describe_and_clean_non_numeric_entries(df):...
311
312
313 > def convert_column_to_numeric(df):...
338
339
340 # This is applied to regular Object Columns
341 > def convert_common_alias_nulls(df):...
380
381
382 > def display_most_common_unique_entries(df, max_display=40):...
400
```

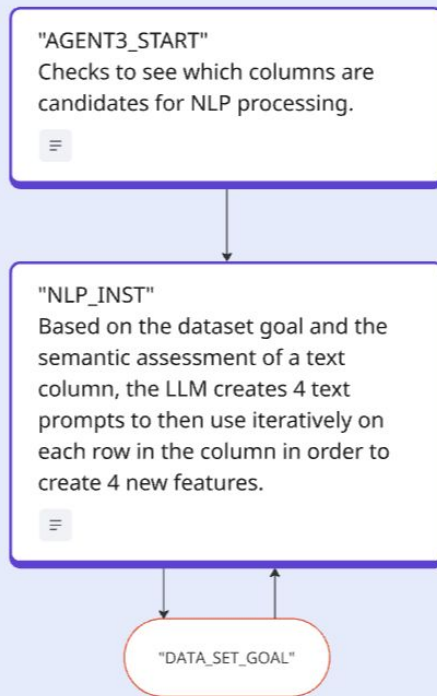
Agent 2 (Outlier & Impute)



Agent 2 Continued...



Agent 3 (NLP Feature Generator)



Agent 5 (Numeric Feature Generator)

"SUPER_AGENTS5_START"

Stronger GPT version that performs parts-of-a-whole search on all numeric features, categorizing them into "POW Groups".

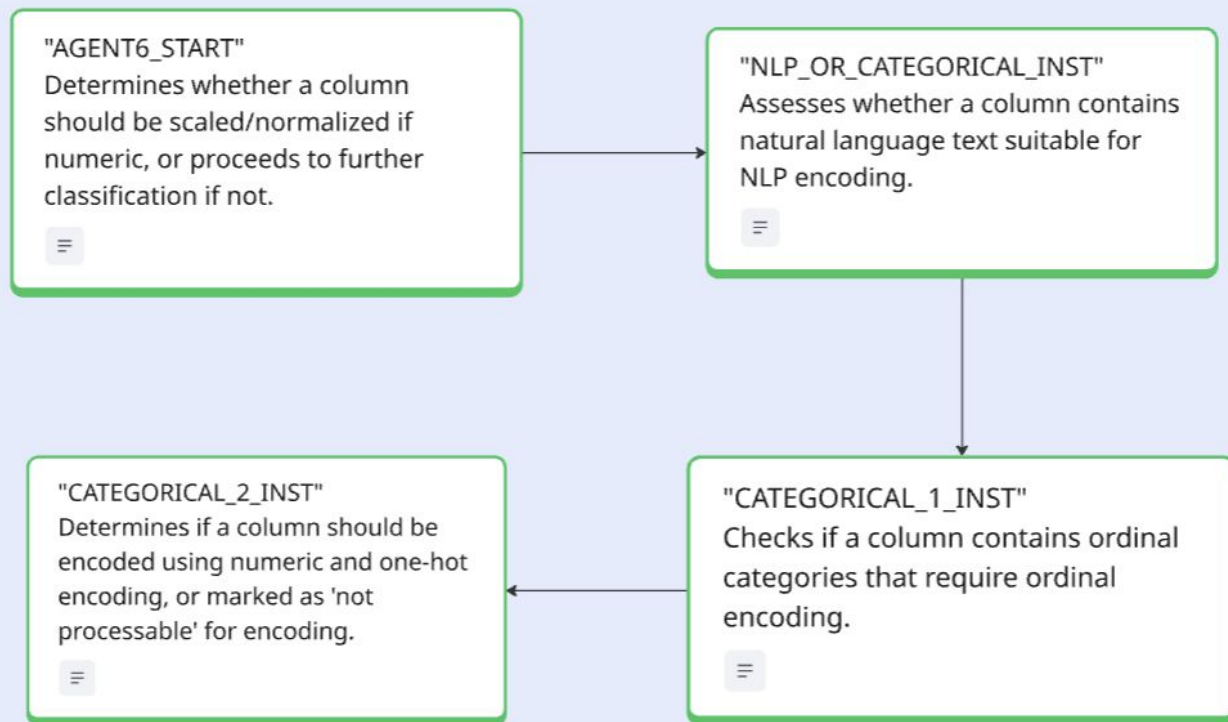


"AGENTS5_START"

Iteratively generates new numeric features from the generated POW Groups.



Agent 6 (Encoding Agent)



ARTI 350: auto-mpg dataset preprocessing

- ***mpg*** | cylinders | displacement | horsepower | weight | acceleration | model year | origin | car name
- Drop name column
- Use medium imputing for question mark nulls in horsepower column
- Bin and one-hot origin column (america, asia, europe)
- MinMax normalizing across numeric predictor columns

ARTI 350: auto-mpg dataset model metrics

		*					*					
	Model	Train R2	Train MSE	Train RMSE	Train MAE	Train ADJR2	Test R2	Test MSE	Test RMSE	Test MAE	Test ADJR2	Overfit (R2)
5	Random Forest Regressor	0.982334	1.112653	1.054824	0.721621	0.981844	0.880812	6.529007	2.555192	1.793700	0.870334	10.152146
7	Extra-Trees	1.000000	0.000006	0.002353	0.000302	1.000000	0.880436	6.549627	2.559224	1.807720	0.869924	11.956427
6	Gradient Boosting	0.974927	1.579109	1.256626	0.985247	0.974233	0.875774	6.804998	2.608639	1.830306	0.864853	9.915341
8	Support Vector Regression	0.766446	14.709481	3.835294	2.709714	0.759981	0.858991	7.724357	2.779273	2.025122	0.846594	-9.254497
1	Ridge Regression	0.806681	12.175455	3.489335	2.657849	0.801329	0.851906	8.112444	2.848235	2.205573	0.838887	-4.522564
0	Linear Regression	0.816403	11.563135	3.400461	2.663928	0.811321	0.844232	8.532831	2.921101	2.251757	0.830538	-2.782911
2	Lasso Regression	0.482999	32.561276	5.706249	4.552270	0.468687	0.571010	23.499706	4.847650	3.941592	0.533296	-8.801097
3	LARS Lasso	0.482993	32.561659	5.706282	4.552310	0.468681	0.571003	23.500077	4.847688	3.941645	0.533289	-8.801029
4	Elastic-Net	0.377305	39.217947	6.262423	5.124353	0.360068	0.448417	30.215200	5.496835	4.660251	0.399926	-7.111197

AMLE Model Metrics (Without POW Features)

		✱					✱					
	Model	Train R2	Train MSE	Train RMSE	Train MAE	Train ADJR2	Test R2	Test MSE	Test RMSE	Test MAE	Test ADJR2	Overfit (R2)
6	Gradient Boosting	0.973070	1.696093	1.302341	1.010626	0.972420	0.887482	6.163617	2.482663	1.773970	0.878921	8.558746
7	Extra-Trees	1.000000	0.000011	0.003387	0.000490	1.000000	0.880462	6.548167	2.558939	1.796970	0.871367	11.953753
5	Random Forest Regressor	0.982848	1.080278	1.039364	0.714567	0.982434	0.880435	6.549674	2.559233	1.809240	0.871337	10.241279
8	Support Vector Regression	0.768091	14.605854	3.821761	2.708886	0.762493	0.858991	7.724339	2.779269	2.023975	0.848262	-9.089994
1	Ridge Regression	0.803648	12.366433	3.516594	2.658646	0.798909	0.853101	8.046976	2.836719	2.168835	0.841924	-4.945307
0	Linear Regression	0.811462	11.874309	3.445912	2.655199	0.806911	0.849160	8.262887	2.874524	2.204689	0.837683	-3.769773
2	Lasso Regression	0.470335	33.358846	5.775712	4.601141	0.457550	0.551898	24.546602	4.954453	4.032618	0.517804	-8.156339
3	LARS Lasso	0.470331	33.359070	5.775731	4.601163	0.457546	0.551894	24.546815	4.954474	4.032640	0.517799	-8.156306
4	Elastic-Net	0.355469	40.593192	6.371279	5.232671	0.339912	0.417155	31.927720	5.650462	4.781910	0.372808	-6.168554

AMLE Model Metrics (With POW Features)

		★					★					
	Model	Train R2	Train MSE	Train RMSE	Train MAE	Train ADJR2	Test R2	Test MSE	Test RMSE	Test MAE	Test ADJR2	Overfit (R2)
6	Gradient Boosting	0.975303	1.555439	1.247172	0.943425	0.974531	0.895641	5.716684	2.390959	1.710810	0.885205	7.966192
7	Extra-Trees	1.000000	0.000012	0.003476	0.000403	1.000000	0.887341	6.171330	2.484216	1.752970	0.876076	11.265831
5	Random Forest Regressor	0.982363	1.110772	1.053932	0.734074	0.981812	0.881553	6.488425	2.547239	1.813580	0.869708	10.081050
8	Support Vector Regression	0.767601	14.636743	3.825800	2.708503	0.760338	0.859068	7.720105	2.778508	2.022670	0.844975	-9.146767
1	Ridge Regression	0.805619	12.242338	3.498905	2.674188	0.799544	0.850796	8.173280	2.858895	2.181557	0.835875	-4.517700
0	Linear Regression	0.829568	10.734004	3.276279	2.540279	0.824242	0.843219	8.588313	2.930582	2.236123	0.827541	-1.365150
2	Lasso Regression	0.470335	33.358846	5.775712	4.601141	0.453783	0.551898	24.546602	4.954453	4.032618	0.507088	-8.156339
3	LARS Lasso	0.470331	33.359070	5.775731	4.601163	0.453779	0.551894	24.546815	4.954474	4.032640	0.507084	-8.156306
4	Elastic-Net	0.355469	40.593192	6.371279	5.232671	0.335328	0.417155	31.927720	5.650462	4.781910	0.358870	-6.168554